

Lab 4: Arrays and Pointers

Matteo Fasiolo

Homework 1: working with arrays

Open the file `arrays.c` and complete the following tasks.

A) Complete the function

```
double maximum(const double values[], int length);
```

The function should:

1. initially treat `values[0]` as the largest value;
2. inspect the remaining elements;
3. return the largest value.

Test it in `main()` using the array provided in the file.

B) Complete the function

```
void swap_first_two(double values[], int length);
```

- If `length >= 2`, swap `values[0]` and `values[1]`.
- Otherwise, leave the array unchanged.

Test it in `main()` using the array provided in the file.

Which array-function pattern does this illustrate? (see the slides)

C) Given the matrix

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$$

stored a row-major integer array, complete the function

```
void print_matrix(const int values[], int nrow,  
                 int ncol);
```

and test that it prints the test matrix in `arrays.c` correctly.

Homework 2: working with pointers

Open the file `pointers.c`.

A) Complete the function

```
int range(const double values[], int length,  
          double *minimum, double *maximum);
```

The function should find the smallest and largest elements of values and store the results at the addresses supplied through minimum and maximum.

It should return 1 after successfully producing both results.

The function should return 0 without accessing the array when:

- `length <= 0;`
- `values == NULL;`
- `minimum == NULL` or `maximum == NULL`.

Test the function using the array provided in `main()`.

While stopped inside `range`, identify which variables in `main()` the pointer parameters `minimum` and `maximum` point to.

B) In the same file, complete

```
void swap_min_max(double values[], int length);
```

The function should:

1. use two pointers to remember the locations of the smallest and largest elements;
2. scan the remaining elements using `values + i` or `&values[i]`;
3. swap the selected elements by dereferencing the two pointers.

If values == NULL or length < 2, leave the array unchanged.

If the minimum or maximum occurs more than once, use its first occurrence.

For the array supplied in main(), the result should be

```
Before: 4.20 -1.00 3.80 7.10 2.40
```

```
After:  4.20 7.10 3.80 -1.00 2.40
```

Use the debugger to inspect minimum, maximum, *minimum and *maximum immediately before the swap.

Submission

Upload your solutions to homeworks 1 and 2 in two separate .c files.

NOTE Blackboard allows uploading multiple files, so this should work fine.

Before submitting, make sure your code compiles and runs!